# GMM


## Ram Prasad


## 5/19/2025

## PPNR Markets

GMM is useful for estimating non-linear models, and is robust to the distributional assumptions required for MLE and heteroskedasticity that may preclude the use of OLS. Statsmodels's GMM implementation is under development, hence I provide portable code for 2-step GMM estimation, standard errors, and hypothesis testing using Hansen's J-test, with two worked through examples:

1) Simulated $N(\mu, \sigma^2)$ data, using mean, variance, skewness, and kurtosis as moment conditions.

2) Estimate parameters of the SDF, using historical returns, consumption data, and the consumption Euler equations for 10 portfolios as moment conditions.

For a full treatment of GMM, see Cochrane (2005) Ch. 10-11. I outline the procedural steps below.

**Step 0**:

Construct moment conditions $f_t(\theta)$, where f is vector-valued, dimension m x 1, such that $E[f_t(\theta)] = 0$. Define the sample average $g_T(\theta) = \frac{1}{T}\sum_{t=1}^{T} f_t(\theta)$.

**Step 1**:

Obtain $\hat{\theta}_1$ and $\hat{S}$

$$W_{\text{initial}} = I_{mxm}$$
$$\hat{\theta}_1 = \arg\min_{\theta}[g_T(\theta)]' W_{\text{initial}}[g_T(\theta)]$$

$$\hat{S} = \frac{1}{T}\sum_{t=1}^{T}[f_t(\hat{\theta}_1)][f_t(\hat{\theta}_1)]'$$

**Step 2**:

Obtain $\hat{\theta}_2$ using $W = \hat{S}^{-1}$

$$\hat{\theta}_2 = \arg\min_{\theta}[g_T(\theta)]' \hat{S}^{-1}[g_T(\theta)]$$

with standard errors given by:

$$\hat{D} = \frac{\partial}{\partial \theta'}\frac{1}{T}\sum_{t=1}^{T} f_t(\hat{\theta}_2)$$
$$\widehat{\text{a.var}}(\hat{\theta}_2) = \frac{1}{T}(\hat{D}'\hat{S}^{-1}\hat{D})^{-1}$$

When $m > p$, for $m$ moment conditions and $p$ parameters, the model is overidentified. Hansen's J-test statistic for overidentifying restrictions is asymptotically Chi-Square, with $m - p$ degrees of freedom:

$$J = T[g_T(\hat{\theta}_2)]'\hat{S}^{-1}[g_T(\hat{\theta}_2)]$$
$$J \xrightarrow{d} \chi^2_{m-p}$$

# GMM Code

I present code for 2-step GMM estimation below. The code requires functions for the moment conditions and standard errors. The latter requires the Jacobian, which can be derived analytically or computed numerically. Generally, the challenge with GMM is formulating the moment conditions. The optimization method (e.g. Nelder-Mead) and associated options are passed in as keyword arguments for maximum flexibility. Step 2 requires inverting $\hat{S}$. The code checks for an ill-conditioned matrix, and will warn the user accordingly, but will proceed unless $\hat{S}$ is singular and numpy throws an error when inverting.

In [1]:
```
import pandas as pd
import numpy as np
import statsmodels.api as sm
import matplotlib.pyplot as plt
import warnings

from scipy import stats
from scipy.optimize import minimize, fsolve
```

In [2]:
```
def gmm(data, moments, se, initial_guess, T=None, num_params=None, W=None, **kwargs):
    """
    Implements GMM for parameter estimation.
    Args:
        data (pd.DataFrame): The data to be used in the moment conditions. T rows.
        moments (callable): A function that takes data and param vector as input and
                            returns a vector of moment conditions.
        se (callable): A function that takes data and param vector, returns standard er
                       for step 2 gmm estimates.
        initial_guess (np.ndarray): An initial guess for the parameter vector.
        T (int): the number of time observations
        num_params (int): the number of parameters to estimate
        W (np.ndarray, optional): The weighting matrix. If None (default), use identify
                                  Efficient GMM requires a consistent estimator of inve
                                  sample moments covariance matrix.
        **kwargs: Additional keyword arguments to pass to minimize function
    Returns:
        dict: a dictionary containing optimization results

    Perform Hansen's J-Test for overidentifying restrictions.
    H0: overidentifying restrictions are valid (model correctly specified)
    Ha: overidentfiying restrictions are not valid (model misspecified)

    If model exactly identified, J-stat should be 0 and parameter estimates shouldn't
    change between Steps 1 and Steps 2.
```

```python
"""
#Define GMM objective function
def objective_function(params, data, moments, W):
    """
    Compute GMM objective function g_T(theta)' W g_T(theta), for g_T(theta)
    the time average of moments f_t(theta)
    Args:
        See documentation above.
    Returns:
        obj (float): g.T @ g if W not provided or g.T @ W @ g if W is provided.
    """
    #Call passed in moments function. f is m x T.
    f = moments(data, params).T

    #Compute the time average. g is m x 1.
    g = np.mean(f, axis=1)
    g = g.reshape(len(g),1)

    #If no weight matrix provided, use identity matrix
    if W is None:
        obj = g.T @ g

    #Otherwise use g' @ W @ g
    else:
        obj = g.T @ W @ g

    #Return objective function value
    return obj[0][0]

#If T unspecified, infer from passed in variables
if T is None:

    #Data should be a T x n array
    T = data.shape[0]

#If length of theta unspecified, infer
if num_params is None:

    #Use length of initial guess
    num_params = len(initial_guess)

#If no W matrix provided, perform 2 step GMM
if W is None:

    #Step 1 uses W = I (identity matrix)
    result = minimize(objective_function,
                      initial_guess,
                      args=(data, moments, W), **kwargs)

    #Store Step 1 parameters estimates
    step_1_params = result.x
    print(f"Step 1 GMM Estimates: {step_1_params}")

    #Compute weighting matrix using covariance of sample moments
    f_t_theta1 = moments(data, step_1_params).T
    S_hat = (1/T)*(f_t_theta1 @ f_t_theta1.T)


    #Store number of moment conditions
    m = S_hat.shape[0]
```

```
#Check invertibility of sample cov matrix
try:

    #use inv(S) as optimal weight
    W_optimal = np.linalg.inv(S_hat)

    """
    Check is S_hat is ill-conditioned. Use the ratio
    ||S_hat||/k(S_hat) for k(S_hat)=||S_hat||||S_hat^-1||
    with the 2 norm, which measures distance between
    S_hat and the nearest singular matrix:
    min{||S_hat - Sing.||} is ||S_hat||/k(S_hat).
    """
    #Compute condition number of S hat
    cond_S_hat = np.linalg.cond(S_hat, p=2)

    #Compute 2 norm of S hat
    S_hat_pnorm = np.linalg.norm(S_hat, ord=2)

    #Compute ratio
    ic_ratio = S_hat_pnorm // cond_S_hat

    #Check if ratio close to 0
    if ic_ratio < 1e-10:
        print("\nWarning: Est. S matrix near singular.")

    #Set flag for computation of SE
    compute_se = True

#Catch error
except np.lingalg.LinAlgError as l:

    #If S is singular, use identity matrix
    print(f"Warning: {f} when inverting S. Using I (1-Step GMM), No SE or J-sta
    W_optimal = np.eye(m)

    #Set flag for computation of SE
    compute_se = False


#Step 2 (efficient GMM) using inverse(S) as weighting matrix
result_efficient = minimize(objective_function,
                            step_1_params,
                            args=(data, moments, W_optimal), **kwargs)
step_2_params = result_efficient.x
print(f"\nStep 2 GMM Estimates: {step_2_params}")

#Compute Step 2 SE
if compute_se:

    #Call passed in function se
    result_se = se(data, step_2_params, W_optimal)

    #Comput J-stat and p-value
    J = T*result_efficient.fun

    #Test stat is Chi-Sq w/m-p D.F.
    p = 1 - stats.chi2.cdf(J, df=m-num_params)
else:
```

```
            result_se = [np.nan for i in range(len(step_2_params))]
            J, p = np.nan, np.nan



    #If W provided, use it
    else:
        result = minimize(objective_function,
                          initial_guess,
                          args=(data, moments, W),
                          **kwargs)

    return {'Estimates':step_2_params,
            'S.E.': result_se,
            "Hansen's J-Stat.":J,
            "P-Value":p,
            "Conc.": "Reject H0" if p < 0.05 else "Fail to Reject H0",
            'Objective Fn. Value':result_efficient.fun,
            'Optimizer Success':result_efficient.success}
```

# Simulation Example

Draw a vector of size T from a $N(\mu, \sigma^2)$ distribution, and use 4 moment restrictions

$$f_t(\mu, \sigma^2) = \begin{pmatrix} X_t - \mu \\ (X_t - \mu)^2 - \sigma^2 \\ (X_t - \mu)^3 \\ (X_t - \mu)^4 - 3\sigma^4 \end{pmatrix}$$

for p=2 parameters to test the code's functionality. If $m = p$, GMM collapses to the method of moments and the second step is superfluous. I compute the Jacobian analytically:

$$\frac{\partial f_t(\mu, \sigma^2)}{\partial \theta'} = \begin{pmatrix} -1 & 0 \\ -2(X_t - \mu) & -2\sigma \\ -3(X_t - \mu)^2 & 0 \\ -4(X_t - \mu)^3 & -12\sigma^3 \end{pmatrix}$$

In [3]:

```
#Simulate data. Set mean, std dev, and T for sims:
sim_mu, sim_sigma, sim_T = 5.0, 2.0, 1000
sim_data = pd.DataFrame(np.random.normal(loc=sim_mu,scale=sim_sigma,size=(sim_T,1)))
sim_data.columns = [f"X{i+1}" for i in sim_data.columns]

#Moment condition for mean and variance. Use 4 conditions to test overidentification.
def sim_mom(data, params):

    #Extract parameters
    mu, sigma = params

    #Moment vectors are Tx1
    #Condition 1 is E(x - mu) = 0
    m1 = data - mu
```

```
    #Condition 2 is E( (x-mu)**2 - sigma^2) = 0
    m2 = (data-mu)**2 - sigma**2

    #Condition 3 is E( (x-mu)**3 ) = 0
    m3 = (data-mu)**3

    #Condition 4 is E( (x - mu)**4  - 3*sigma**4) = 0
    m4 = (data-mu)**4 - 3*(sigma**4)

    #Stack and return a Txm array
    m = np.hstack([m1,m2,m3,m4])

    return m


#Standard errors for Step 2 GMM estimates - MoM N(mu, sigma^2) ex
def analytic_se(data, params, W):

    #Extract parameters
    mu, sigma = params

    #Closed form for Jacobian. 4x2 matrix.
    D = np.array([[-1.0,                                  0.0],
                  [-2*(data - mu).mean(),            -2*sigma],
                  [-3*((data-mu)**2).mean(),              0.0],
                  [-4*((data-mu)**3).mean(),    -12*sigma**3]])

    #Asymptotic covariance
    T = len(data)
    a_cov = (1/T)*np.linalg.inv(D.T @ W @ D)

    #Compute SE
    se = np.diag(a_cov)**(1/2)
    return se


#Initial parameter guess
initial_params_sim = np.array([0.0,1.0])

#set bounds for strictly positive std dev
bounds_sim = [(None, None), (0, None)]

#Call GMM fn. Use Nelder Mead optimization, pass in tolerances.
gmm_results_sim = gmm(data=sim_data.values,
                      moments=sim_mom,
                      se=analytic_se,
                      initial_guess=initial_params_sim,
                      method='Nelder-Mead',
                      bounds = bounds_sim,
                      options={'fatol':1e-30, 'xatol':0.0001})

#gmm(data, moments, se, initial_guess, T=None, num_params=None, W=None, **kwargs)
print(f"\nTrue Parameter Values: {sim_mu, sim_sigma}")
print("\nEfficient GMM results:\n"+"-"*25)
for k,v in gmm_results_sim.items():
    try:
        print(k,":",np.round(v,3))
    except:
        print(k,":",v)
```

```
Step 1 GMM Estimates: [5.00657389 2.01729651]

Step 2 GMM Estimates: [5.01035739 1.98566366]

True Parameter Values: (5.0, 2.0)

Efficient GMM results:
-------------------------
Estimates : [5.01  1.986]
S.E. : [0.063 0.045]
Hansen's J-Stat. : 0.542
P-Value : 0.763
Conc. : Fail to Reject H0
Objective Fn. Value : 0.001
Optimizer Success : True
```

The function works as intended. I proceed with a second example of non-linear parameter estimation from asset pricing .

# Background

### Pricing Equation

I derive the pricing equation. Following notation from Cochrane (2005), let:

- $x_{t+1}$ be a payoff at t+1. A time t, the payoff is a random variable.
- $U(c_t, c_{t+1})$ be an investor's utility function over two periods as a function of consumption in each period.
- $c_{t+1}$ is consumption at t+1. At time t, this is also a random variable.
- $u(\cdot)$ is a one period utility function. Utility is concave (increasing to capture higher utility from more consumption, but at a decreasing rate to capture declining marginal utility of additional consumption).
- $\beta$ is the subjective discount factor, which captures an investor's preference for consumption today over deferred consumption.
- $p_t$ is the price of the payoff $x_{t+1}$. For a stock, payoff $x_{t+1}$ can be price appreciation plus a dividend: $x_{t+1} = p_{t+1} + d_{t+1}$.

A representative investor has utility function:

$$U(c_t, c_{t+1}) = u(c_t) + \beta E_t[u(c_{t+1})]$$

and can buy or sell an unrestricted amount $\epsilon_t$ of the payoff $x_{t+1}$ at price $p_t$. Assume the investor has original consumption $e_t$ and solves the two-period optimization problem:

$$\begin{aligned} &\max_{\epsilon} u(c_t) + E_t[\beta u(c_{t+1})] \\ &\text{s.t.} \\ &c_t = e_t - p_t\epsilon \\ &c_{t+1} = e_{t+1} + x_{t+1}\epsilon \end{aligned}$$

The first order condition is:

$$\begin{aligned}
\max_{\epsilon}\ u(e_t - p_t\epsilon) + E_t[\beta u(e_{t+1} + x_{t+1}\epsilon)] \\
\frac{d}{d\epsilon}U(\epsilon) = 0 \\
-p_t u'(e_t - p_t\epsilon) + E_t[\beta u'(e_{t+1} + x_{t+1}\epsilon)x_{t+1}] = 0 \\
p_t u'(c_t) = E_t[\beta u'(c_{t+1})x_{t+1}] \\
p_t = E_t\left[\beta\frac{u'(c_{t+1})}{u'(c_t)}x_{t+1}\right]
\end{aligned}$$

At t, $p_t$ and $c_t$ are non-random. Let $m_{t+1}$ be the stochastic discount factor and $R_t$ the gross return:

$$\begin{aligned}
m_{t+1} &\equiv \beta\frac{u'(c_{t+1})}{u'(c_t)} \\
R_{t,t+1} &\equiv \frac{x_{t+1}}{p_t} \\
1 &= E_t[m_{t+1}\frac{x_{t+1}}{p_t}] \\
1 &= E_t[m_{t+1}R_{t,t+1}]
\end{aligned}$$

## GMM

Assume power utility (CRRA). The Arrow-Pratt coefficient of relative risk aversion is $\gamma$:

$$\begin{aligned}
u(c_t) &= \frac{c_t^{1-\gamma}}{1-\gamma} \\
u'(c_t) &= c_t^{-\gamma} \\
u''(c_t) &= -\gamma c_t^{-\gamma-1} \\
A &\equiv -\frac{u''(c_t)}{u'(c_t)}c_t \\
&= \gamma \\
1 &= E_t\left[\beta\frac{c_{t+1}^{-\gamma}}{c_t^{-\gamma}}R_{t,t+1}\right]
\end{aligned}$$

For a set of $m$ test assets, indexed by $i$, natural moment conditions are:

$$E_t\left[\beta\left(\frac{c_{t+1}}{c_t}\right)^{-\gamma}R_{t,t+1}^i - 1\right] = 0$$

Let

$$f_t(\theta) = \begin{bmatrix} \beta\left(\frac{C_{t+1}}{C_t}\right)^{-\gamma}R_{t,t+1}^1 \\ \vdots \\ \beta\left(\frac{C_{t+1}}{C_t}\right)^{-\gamma}R_{t,t+1}^m \end{bmatrix}_{m\times 1}$$

with sample average:

$$g_T(\theta) = \frac{1}{T}\sum_{t=1}^{T} f_t(\theta)$$

Note, $\frac{d}{d\gamma}a^{-\gamma} = -a^{-\gamma}\ln(a)$. The Jacobian is:

$$\frac{\partial f_t(\beta,\gamma)}{\partial \theta'} = \begin{pmatrix} \left(\frac{c_{t+1}}{c_t}\right)^{-\gamma} R^1_{t,t+1} & -\beta R^1_{t,t+1}\left(\frac{C_{t+1}}{C_t}\right)^{-\gamma}\ln\left(\frac{C_{t+1}}{C_t}\right) \\ \vdots & \vdots \\ \left(\frac{c_{t+1}}{c_t}\right)^{-\gamma} R^m_{t,t+1} & -\beta R^m_{t,t+1}\left(\frac{C_{t+1}}{C_t}\right)^{-\gamma}\ln\left(\frac{C_{t+1}}{C_t}\right) \end{pmatrix}_{mx2}$$

# Dataset Construction

I use historical inflation, return, and consumption data to estimate the SDF's parameters $\beta, \gamma$. All data is quarterly or aggregated from monthly to quarterly.

- Quarterly inflation rate. Use CPIAUCSL from FRED, CPI (price level) is monthly and seasonally-adjusted.

$$\pi_t = \frac{\text{CPI}_q}{\text{CPI}_{q-1}} - 1$$
$$\pi_{Q1-2011} = \frac{\text{CPI}_{2011-03}}{\text{CPI}_{2010-12}} - 1$$

See: https://fred.stlouisfed.org/series/CPIAUCSL

- Quarterly, gross, value-weighted real returns on book to market sorted portfolios. Use monthly returns from Ken French's website, and aggregate to quarterly by taking the cumulative product:

$$R^N_{Q1-2011} = R_{2011-01}R_{2011-02}R_{2011-03}$$
$$R_{Q1-2011} = \frac{R^N_{Q1-2011}}{1+\pi_{Q1-2011}}$$

Raw returns are monthly, net (not gross), nominal, and reported in percentage points.

See: https://mba.tuck.dartmouth.edu/pages/faculty/ken.french/Data_Library/det_form_btm.html

- Quarterly, gross, real consumption growth. Use FRED PCE, reported in seasonally-adjusted, annual, $BN.

$$\frac{C_{t+1}^N}{C_t^N} = \frac{\text{PCE}_{t+1}}{\text{PCE}_t}$$

$$\frac{C_{t+1}}{C_t} = \frac{C_{t+1}^N / C_t^N}{1 + \pi_t}$$

See: https://fred.stlouisfed.org/series/PCEC

Ideally, I'd use real per-capita consumption growth calculated from BEA data, specifically NIPA data Table 7.1: line 8 (nondurable goods in current dollars), line 9 (services in current dollars), line 16 (non-durable goods in chained 2012 dollars), line 17 (services in chained 2012 dollars). Values are per-capita expenditures, seasonally adjusted, at annual rates. Divide by 4 to get quarterly values. For ND: non-durables, S: services, g: gross quarterly growth rates in real terms, e: expenditures in nominal terms, quarterly consumption growth is:

$$\left(\frac{e_{q-1}^{ND}}{e_{q-1}^{ND} + e_{q-1}^{S}}\right) g_q^{ND} + \left(\frac{e_{q-1}^{S}}{e_{q-1}^{ND} + e_{q-1}^{S}}\right) g_q^{S}$$

However, the data is currently unavailable.

I first construct the dataset using CSVs. The internal firewall blocks API calls.

In [4]:

```
#Set path to data
data_path = "C:/Users/RP64662/Downloads/"

#CPI data
cpi_fname = "CPIAUCSL_042025.csv"

#Fama-French book to market portfolio returns
beme_fname = "be_me_returns.csv"

#Consumption data
pcec_fname = "PCEC.csv"

#Monthly, SA price level from FRED
cpi_fred = pd.read_csv(data_path+cpi_fname)
cpi_fred = cpi_fred.dropna(how="all",axis=1).dropna(how="all",axis=0)

#Convert observation date col (YYYY-MM-DD) to quarterly, using last month's obs as end
cpi_fred["Date"] = pd.to_datetime(cpi_fred["observation_date"])
cpi_fred = cpi_fred.drop(columns=["observation_date"])
cpi_fred = cpi_fred.set_index("Date").resample("Q").aggregate("last")

#Compute inflation
cpi_fred["pi(t)"] = cpi_fred["CPIAUCSL"].pct_change(1)

#Return data is monthly, in percentage points
returns = pd.read_csv(data_path+beme_fname)
returns = returns.dropna(how="all",axis=1).dropna(how="all",axis=0)

#Format Date from YYYYMM to date time
returns["Date"] = pd.to_datetime(returns["Date"].apply(lambda x: str(x)[0:4]+"-"+str(x)

#Convert to gross returns in %, take cumulative product of monthly to get quarterly
```

```
returns = returns.set_index("Date")
returns = 1.0 + returns/100.00
returns = returns.resample("Q").aggregate("prod")

#Rename columns
returns.columns = [f"R_{col}" for col in returns.columns]

#Quarterly, SA PCE from FRED
pce_fred = pd.read_csv(data_path+pcec_fname)
pce_fred = pce_fred.dropna(how="all",axis=1).dropna(how="all",axis=0)

#Convert observation date col to date time, add end of quarter offset
pce_fred["Date"] = pd.to_datetime(pce_fred["observation_date"]) + pd.tseries.offsets.Qu
pce_fred = pce_fred.set_index("Date").drop(columns=["observation_date"])

#Compute Gross consumption growth. Divide by 4 (data is annual).
pce_fred["C(t)/C(t-1)"] = 1.0 + (pce_fred["PCEC"].pct_change(1))

#Outer merge all dataframes on Date Index
gmm_data = pce_fred.join(cpi_fred, how="outer").join(returns, how="outer")

#Now drop rows with missing observations. SUbset to post WWII.
gmm_data = gmm_data.dropna(how="any", axis=0)

#Compute real returns and real consumption growth
gmm_data[returns.columns] = gmm_data[returns.columns].apply(lambda x: x/(1.0+gmm_data["
gmm_data["C(t)/C(t-1)"] /= (1+gmm_data["pi(t)"])

#Subset data to exclude Covid.
t0 = pd.to_datetime("1951-03-31")
t1 = pd.to_datetime("2019-12-31")
gmm_data = gmm_data[(gmm_data.index >= t0) & (gmm_data.index <= t1)]
print("Return and consumption growth data for GMM exercise:")
display(gmm_data)
```

Return and consumption growth data for GMM exercise:

| Date | PCEC | C(t)/C(t-1) | CPIAUCSL | pi(t) | R_Decile1 | R_Decile2 | R_Decile3 | R_Decile4 | R_Decile5 |
|---|---|---|---|---|---|---|---|---|---|
| **1951-03-31** | 209.207 | 1.020135 | 25.880 | 0.036029 | 1.020913 | 1.044708 | 1.030644 | 1.001699 | 1.024529 |
| **1951-06-30** | 204.942 | 0.977725 | 25.930 | 0.001932 | 1.020057 | 0.966831 | 0.977981 | 0.996064 | 1.094679 |
| **1951-09-30** | 207.616 | 1.009156 | 26.030 | 0.003857 | 1.103889 | 1.127832 | 1.124744 | 1.113361 | 1.105242 |
| **1951-12-31** | 211.590 | 1.002200 | 26.470 | 0.016904 | 0.994641 | 0.994440 | 0.989746 | 1.013669 | 1.001783 |
| **1952-03-31** | 212.969 | 1.009569 | 26.390 | -0.003022 | 1.018307 | 1.040207 | 1.035044 | 1.044681 | 1.026380 |
| **...** | ... | ... | ... | ... | ... | ... | ... | ... | ... |
| **2018-12-31** | 14099.199 | 1.004597 | 252.767 | 0.002320 | 0.841522 | 0.826539 | 0.894345 | 0.875615 | 0.856429 |

| Date | PCEC | C(t)/C(t-1) | CPIAUCSL | pi(t) | R_Decile1 | R_Decile2 | R_Decile3 | R_Decile4 | R_Decile5 |
|---|---|---|---|---|---|---|---|---|---|
| **2019-03-31** | 14150.191 | 0.997657 | 254.277 | 0.005974 | 1.164081 | 1.153314 | 1.112049 | 1.124838 | 1.137859 |
| **2019-06-30** | 14350.662 | 1.010448 | 255.213 | 0.003681 | 1.038861 | 1.048314 | 1.024717 | 1.035542 | 1.048006 |
| **2019-09-30** | 14548.282 | 1.008960 | 256.430 | 0.004769 | 0.999796 | 1.013692 | 0.999717 | 1.007791 | 1.000068 |
| **2019-12-31** | 14701.038 | 1.001904 | 258.630 | 0.008579 | 1.085823 | 1.121958 | 1.070918 | 1.080658 | 1.054629 |

276 rows × 14 columns

Perform data quality checks:

In [5]:
```
#DF NaN counts and dtypes
display(gmm_data.info())

#Check outliers using Z-scoring and staleness.
zs_data = gmm_data[[col for col in gmm_data.columns if gmm_data[col].dtype==float]]
zs_data = zs_data.apply(lambda x: (x-x.mean())/x.std(ddof=1))
outliers = zs_data.apply(lambda x: np.abs(x)>5).sum()/len(zs_data)

#Staleness
staleness = (np.abs(gmm_data.diff(1)/gmm_data.shift(1)) < 1e-5).sum()/len(gmm_data)

#Combine DQ checks into one DF
dq_df = pd.DataFrame(data={"Outliers":outliers, "Staleness":staleness}).T
print("DQ checks for outliers and staleness:")
display(dq_df)
```

```
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 276 entries, 1951-03-31 to 2019-12-31
Data columns (total 14 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PCEC         276 non-null    float64
 1   C(t)/C(t-1)  276 non-null    float64
 2   CPIAUCSL     276 non-null    float64
 3   pi(t)        276 non-null    float64
 4   R_Decile1    276 non-null    float64
 5   R_Decile2    276 non-null    float64
 6   R_Decile3    276 non-null    float64
 7   R_Decile4    276 non-null    float64
 8   R_Decile5    276 non-null    float64
 9   R_Decile6    276 non-null    float64
 10  R_Decile7    276 non-null    float64
 11  R_Decile8    276 non-null    float64
 12  R_Decile9    276 non-null    float64
 13  R_Decile10   276 non-null    float64
dtypes: float64(14)
memory usage: 32.3 KB
None
DQ checks for outliers and staleness:
```

| | PCEC | C(t)/C(t-1) | CPIAUCSL | pi(t) | R_Decile1 | R_Decile2 | R_Decile3 | R_Decile4 | R_Decile5 | R |
|---|---|---|---|---|---|---|---|---|---|---|
| **Outliers** | 0.0 | 0.0 | 0.000000 | 0.003623 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | |
| **Staleness** | 0.0 | 0.0 | 0.021739 | 0.000000 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | |

There are no missing observations, but there is minor staleness in price level data, which is to be expected. I proceed with estimation.

# Estimation and Inference

I define functions for the moment conditions and standard errors below.

In [6]:
```
#Set column names for consumption and returns
cons_col = "C(t)/C(t-1)"
R_cols = returns.columns

#Moment condition for mean and variance
def euler_mom(data, params, cons_col=cons_col, R_cols=R_cols):
    """
    Moment conditions based upon consumption Euler equation.
    Args:
        data (pd.DataFrame): DF containing time series of returns and consumption growt
        params (np.array): vector containing SDF parameters
        cons_col (str): string name of data's consumption growth column
        R_cols (list): list of string names for data's gross return columns
    Output:
        m (np.array): a T x m array containing time obs (row-wise) of moments (col-wise
    """

    #Extract parameters
    beta, gamma = params

    #Extract consumption growth and returns in array form.
    #Cons. growth is Tx1 and n Gross returns form a T x n array
    c_growth = data[[cons_col]].values
    R = gmm_data[R_cols].values

    """
    Multiply R and c^-gamma element by element, then multiply by beta, a scalar,
    and subtract 1. M is dimension T x R for T obs of R returns.
    """
    m = beta*(np.multiply(c_growth**(-gamma), R)) - 1

    #Return sample moment vector
    return m

#Standard errors for Step 2 GMM estimates
def euler_se(data, params, W, cons_col=cons_col, R_cols=R_cols):
    """
    Standard errors for beta and gamma, based upon consumption Euler equation.
    Args:
        data (pd.DataFrame): DF containing time series of returns and consumption growt
        params (np.array): vector containing SDF parameters
```

```
        W (np.array): weight matrix of shape m x m
        cons_col (str): string name of data's consumption growth column
        R_cols (list): list of string names for data's gross return columns
    Output:
        se (np.array): a p x 1 array containing standard errors of parameter estimates
    Use analytic Jacobian instead of numerical solver. m moments, p parameters.
    """

    #Extract paraemters
    beta, gamma = params

    #Do this analytically - just diff wrt beta, alpha
    #Init empty list to hold Jacobian
    D = []

    #Extract consumption growth and returns in array form.
    #Cons. growth is Tx1 and n Gross returns form a T x n array
    c_growth = data[[cons_col]].values
    R = gmm_data[R_cols].values
    T = R.shape[0]

    #Iterate through moment conditions (# of Returns)
    for i in range(R.shape[1]):

        #Reshape gross return asset i to T x 1
        R_i_t = R[:,i].reshape(len(R[:,i]),1)

        #partial deriv of i'th asset condition w.r.t beta
        dj1 = np.multiply((c_growth**-gamma),R_i_t)

        #partial deriv of i'th asset condition w.r.t alpha
        dj2 = np.multiply(-1*beta*R_i_t,np.multiply((c_growth**-gamma),np.log(c_growth)

        #Append 1x2 time average
        D.append([np.mean(dj1), np.mean(dj2)])

    #Stack elements
    D = np.array(D)

    #Asymptotic covariance
    a_cov = (1/T)*np.linalg.inv(D.T @ W @ D)

    #Compute SE
    se = np.diag(a_cov)**(1/2)
    return se


#Initial parameter guess
initial_params_m = np.array([1.0,1.0])

#Call GMM fn. Use Nelder-Mead optimization, pass in tolerances.
gmm_results_m = gmm(gmm_data,
                    euler_mom,
                    euler_se,
                    initial_params_m,
                    method='Nelder-Mead',
                    options={'fatol':1e-30, 'xatol':0.0001,
                            })

print("\nEfficient GMM results:\n"+"-"*25)
```

```
for k,v in gmm_results_m.items():
    try:
        print(k,":",np.round(v,3))
    except:
        print(k,":",v)
```

```
Step 1 GMM Estimates: [ 1.29867383 55.1642693 ]

Warning: Est. S matrix near singular.

Step 2 GMM Estimates: [ 1.19351005 34.33737754]

Efficient GMM results:
-------------------------
Estimates : [ 1.194 34.337]
S.E. : [ 0.176 27.688]
Hansen's J-Stat. : 9.837
P-Value : 0.277
Conc. : Fail to Reject H0
Objective Fn. Value : 0.036
Optimizer Success : True
```

# Discussion of Results

Using 1951 Q1 - 2019 Q4 data (inclusive, T=276),

- $\hat{\beta}_{\text{GMM}} > 1$, meaning preference for future consumption over current consumption. There are behavioral explanations for negative time preference in specific contexts, e.g. "savoring" - a preference for deferred gratification over instant, and "dread", or expediting unpleasant experiences to get them over with, but $\beta$ <1 is consistent with observable economic behavior and the time value of money.

- $\hat{\gamma}_{\text{GMM}}$ indicates an implausibly high level of risk aversion. Mehra and Prescott (1985) argue $\gamma$ is at most 10, and spawned a vast literature attempting to explain the equity risk premium puzzle. Reasonable levels of risk aversion imply a much smaller equity risk premium than that which was historically observed. High $\gamma$ is a well-known deficit of standard CRRA models.

I ran the exercise in Stata and found similar results, so I am confident in the code provided.

Quickly checking GMM parameter estimates is challenging. If the model was linear and endogeneity wasn't a concern, or I could make distributional assumptions about consumption growth and returns, I'd use OLS or MLE. One option is to use a Taylor series expansion of the non-linear moment condition, but this is an approximation.

Log-linearization is an alternative, which I demonstrate below. For log-normally distributed $X$ with parameters $\mu, \sigma$, a useful property is that:

$$\ln\left(E(X)\right) = E\left(\ln(X)\right) + \frac{1}{2}\text{Var}\left(\ln(X)\right)$$
$$\ln\left(E(X)\right) \approx E\left(\ln(X)\right)$$

The approximation holds when X is close to its expected value and therefore variance is negligible. The case for discarding the variance term must be economic. Given uncertainty regarding

consumption growth and returns, I can't simply discard the variance. I instead assume consumption growth and log returns are jointly conditional log normal, meaning conditioning on all information at t, the product $m_{t+1}R^i_{t,t+1}$ is log normally distributed.

$$
\begin{aligned}
E_t\left[\beta\left(\frac{c_{t+1}}{c_t}\right)^{-\gamma}R^i_{t,t+1}-1\right] &= 0 \\
\ln\left(E_t\left[\beta\left(\frac{c_{t+1}}{c_t}\right)^{-\gamma}R^i_{t,t+1}\right]\right) &= \ln(1) \\
E_t\left(\ln(\beta)-\gamma\ln(\frac{C_{t+1}}{C_t})+\ln(R^i_{t,t+1})\right) &= -\frac{1}{2}\mathrm{Var}_t\left(\ln(\beta)-\gamma\ln(C_{t+1}/C_t)+\ln(R^i_{t,t+1})\right) \\
\ln(\beta)-\gamma E_t\left(\ln(\frac{C_{t+1}}{C_t})\right)+E_t\left(\ln(R^i_{t,t+1})\right) &= -\frac{1}{2}\mathrm{Var}_t\left(\gamma\ln(C_{t+1}/C_t)-\ln(R^i_{t,t+1})\right) \\
&= -\frac{1}{2}\left[\gamma^2\mathrm{Var}_t\left(\ln(\frac{C_{t+1}}{C_t})\right)+\mathrm{Var}_t\left(\ln(R^i_{t,t+1})\right)-\right.
\end{aligned}
$$

Introduce notation for variance and covariance terms to simplify:

$$
\begin{aligned}
\sigma^2_{\Delta c} &\equiv \mathrm{Var}_t\left(\ln(\frac{C_{t+1}}{C_t})\right) \\
\sigma^2_{R^i} &\equiv \mathrm{Var}_t\left(\ln(R^i_{t,t+1})\right) \\
\sigma_{\Delta c R^i} &\equiv \mathrm{Cov}_t\left(\ln(\frac{C_{t+1}}{C_t}),\ln(R^i_{t,t+1})\right)
\end{aligned}
$$

For a risky asset:

$$
E_t\left(\ln(R^i_{t,t+1})\right) = -\ln(\beta)+\gamma E_t\left(\ln(\frac{C_{t+1}}{C_t})\right)-\frac{1}{2}\gamma^2\sigma^2_{\Delta c}-\frac{1}{2}\sigma^2_{R^i}+\gamma\sigma_{\Delta c R^i}
$$

for the risk free asset, its variance is 0 and its covariance with consumption growth is 0:

$$
\ln(R_f) = -\ln(\beta)+\gamma E_t\left(\ln(\frac{C_{t+1}}{C_t})\right)-\frac{1}{2}\gamma^2\sigma^2_{\Delta c}
$$

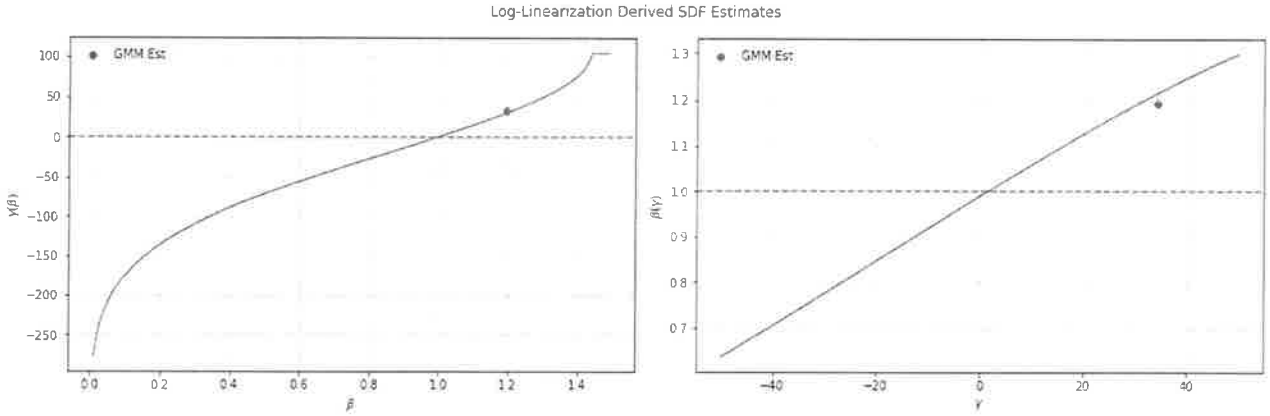
and excess returns are:

$$
E_t\left(\ln(R^i_{t,t+1})-\ln(R_f)\right) = -\frac{1}{2}\sigma^2_{R^i}+\gamma\sigma_{\Delta c R^i}
$$

Estimating $\gamma$ remains a challenge because the terms in the expressions above are conditional expectations and variances, but I only observe realized returns and consumption growth. Even if I assume that expectations are rational (e.g. $\ln(R^i_{t,t+1}) = \ln(R^i_{t-1,t})+\epsilon^i_{t+1}$ such that $E_t\left(\ln(R^i_{t,t+1})\right) = \ln(R^i_{t-1,t})$ and similarly for log consumption growth), the conditional variance and covariance terms need a seprate time series model or an additional assumption of homoskedasticity such that the terms are absorbed by the constant in a regression. Generated regressors and broader endogeneity concerns plague any attempt at regression.

Instead, I use the equation for the risk free asset, solving for $\gamma$ as a function of candidate $\beta$ values and the time series moments of consumption growth:

```
#Compute variance of log consumption growth
sigma_sq_ln_cons = np.var(np.log(gmm_data[["C(t)/C(t-1)"]]))

#Compute average log consumption growth
mu_ln_cons = np.mean(np.log(gmm_data[["C(t)/C(t-1)"]]))

#Define function to compute gamma as function of beta
def gamma_f(gamma, rf, beta, mu_ln_cons=mu_ln_cons, sigma_sq_ln_cons=sigma_sq_ln_cons):
    return np.log(rf) + np.log(beta) - gamma*mu_ln_cons + 0.5*(gamma**2)*sigma_sq_ln_co

#Define function to compute beta as a function of gamma
def beta_f(beta, rf, gamma, mu_ln_cons=mu_ln_cons, sigma_sq_ln_cons=sigma_sq_ln_cons):
    return np.log(rf) + np.log(beta) - gamma*mu_ln_cons + 0.5*(gamma**2)*sigma_sq_ln_co

#grid for beta, gamma
beta_grid = np.arange(0.01,1.5, 0.01)
gamma_grid = np.arange(-50.0,51.0,1.0)

#Derived gamma, beta
with warnings.catch_warnings():
    warnings.simplefilter("ignore")
    derived_gamma = [fsolve(gamma_f, x0=[10.0], args=(1.01,beta)) for beta in beta_grid
    derived_beta = [fsolve(beta_f, x0=[1.0], args=(1.01,gamma)) for gamma in gamma_grid

#Plot
fig, axs = plt.subplots(nrows=1, ncols=2, figsize=(15,5))
gmm_beta, gmm_gamma = gmm_results_m["Estimates"]

#Gammas as function of beta
axs[0].plot(beta_grid, derived_gamma)
axs[0].set_xlabel(r'$\beta$')
axs[0].set_ylabel(r'$\gamma(\beta)$')
axs[0].grid()
axs[0].axhline(y=0.0, linestyle="--")
axs[0].scatter(gmm_beta, gmm_gamma, label="GMM Est.", color="red")
axs[0].legend()

#Beta as function of gamma
axs[1].plot(gamma_grid, derived_beta)
axs[1].set_xlabel(r'$\gamma$')
axs[1].set_ylabel(r'$\beta(\gamma)$')
axs[1].grid()
axs[1].axhline(y=1.0, linestyle="--")
axs[1].scatter(gmm_gamma, gmm_beta, label="GMM Est.", color="red")
axs[1].legend()

plt.suptitle("Log-Linearization Derived SDF Estimates")
plt.tight_layout()
plt.show()
```




The results above confirm that the high $\beta$ and $\gamma$ from GMM are a product of historical low consumption variance and/or covariance with returns, not an issue with the GMM code.

### LN Property

$$
\begin{aligned}
Z &\sim N(0,1) \\
X &\equiv e^{\mu+\sigma Z} \sim \mathrm{LN}(\mu,\sigma) \\
Z &= \frac{\ln(X)-\mu}{\sigma} \\
E(X) &= e^{\mu+0.5\sigma^2} \\
\ln\left(E(X)\right) &= \mu + \sigma E(Z) + 0.5\sigma^2 \mathrm{Var}(Z) \\
&= \mu + \sigma E\left(\frac{\ln(X)-\mu}{\sigma}\right) + 0.5\sigma^2 \mathrm{Var}\left(\frac{\ln(X)-\mu}{\sigma}\right) \\
&= E\left(\ln(X)\right) + 0.5\mathrm{Var}\left(\ln(X)\right)
\end{aligned}
$$

## References:


Cochrane, J.H. (2005) Asset Pricing. 2nd Edition, Princeton University Press, Princeton and Oxford

Mehra, Rajnish; Prescott, Edward C. (March 1985). "The equity premium: A puzzle" (PDF). Journal of Monetary Economics. 15 (2): 145–161